FACTOR

Factual Agentic Content Orchestrator

A self-hosted, multi-agent AI pipeline for autonomous production of hallucination-free articles, reviews, tutorials, and opinion pieces — strictly grounded in verified official sources, scientific journals, and curated book repositories.

Live on AMD Radeon PRO W7900 · proven on AMD Instinct MI300X (ROCm)

Technical Whitepaper · Version 1.2

July 2026

1. Executive Summary

FACTOR (Factual Agentic Content Orchestrator) is an advanced, self-hosted AI agentic pipeline designed to autonomously produce high-quality, hallucination-free content at scale. Unlike conventional AI writing tools that generate text from a model's parametric memory, FACTOR enforces a strict **retrieval-first architecture**: every factual claim in every published piece must be traceable to a verified source in the user's curated knowledge base — official publications, peer-reviewed scientific journals, or licensed book repositories.

FACTOR is **topic-agnostic**. Users define their own content domains — dentistry, personal finance, software engineering, gastronomy, public policy — by creating **Topic Workspaces**, each with its own source connectors, topic backlog, editorial style, and publishing target. A single FACTOR deployment can orchestrate multiple workspaces in parallel, producing multilingual articles, product reviews, step-by-step tutorials, and clearly-labeled opinion pieces, each passing through independent fact-checking and bias-review gates before a single word reaches production.

This whitepaper specifies FACTOR's architecture, agent orchestration model, anti-hallucination gate system, data model, deployment profiles (hybrid and fully self-hosted), and hardware requirements.

2. Problem Statement

Large language models write fluently but not reliably. Left ungrounded, they hallucinate citations, invent statistics, and state outdated facts with full confidence. For organizations publishing in high-stakes domains (health, finance, law — Google's "YMYL" categories) this is not a cosmetic problem: it is a legal, ethical, and reputational one. Meanwhile, manual editorial pipelines cannot match the cadence modern content operations require. FACTOR resolves this tension by making **verifiability a hard architectural constraint** rather than a prompt-engineering suggestion.

3. Design Principles

- **Grounded generation (RAG-first).** Writer agents may only use material retrieved from the workspace corpus. No claim from parametric memory is permitted in factual content types.
- **Citation-or-drop.** Any factual sentence without a resolvable citation to a corpus chunk is rewritten or removed — never silently published.
- **Separation of cognitive roles.** The agent that writes is never the agent that verifies. Fact-checkers run in a clean context with direct access to original sources, so they cannot rationalize the writer's output.
- **Pipeline as a state machine.** Every piece traverses explicit, audited states. A failed gate triggers revision or rejection; nothing ships half-verified.
- **Honest genre labeling.** Reviews and opinion pieces are permitted subjectivity — but premises must still be grounded, and the piece is explicitly labeled as opinion.
- **Self-hosted and sovereign.** All orchestration, data, and (optionally) inference run on the operator's own infrastructure. No content or corpus leaves the deployment unless the operator opts into external APIs.
- **Human-in-the-loop by default, autonomy by merit.** New workspaces start with mandatory human review; autonomy is granted per-workspace once measured pass-rates justify it.

4. System Architecture

FACTOR is a set of containerized services deployed on a self-hosted PaaS (Dokploy or equivalent Docker orchestration). The core stack is Node.js/TypeScript with BullMQ over Redis for orchestration and PostgreSQL with pgvector as the single source of truth for corpus, pipeline state, and published content.

Component	Technology	Role
Orchestrator API	Node.js / TypeScript (NestJS or Fastify)	Workspace & pipeline management, admin UI backend, webhooks, Bull Board
Worker pool	Node.js workers on BullMQ	Ingestion, retrieval, agent steps, image generation, publishing
Queue & cache	Redis + BullMQ	Job queues, retries, repeatable schedules, rate limiting
Database	PostgreSQL + pgvector (HNSW)	Corpus chunks & embeddings, topics, pipeline runs, articles, audit trail
LLM inference	Hosted API / Fireworks AI (hybrid), or Google Gemma 3 via vLLM or llama.cpp on AMD Instinct / Radeon (ROCm, self-hosted)	All cognitive agent steps
Embeddings	Voyage / qwen3-embedding (hybrid) or EmbeddingGemma / bge-m3 on AMD (self-hosted)	Corpus and query vectorization
Image generation	Imagen/gpt-image (hybrid) or SDXL/Flux on AMD ROCm via ComfyUI	Featured images and illustrations
Document parsing	GROBID + unstructured/pdf-parse	Scientific PDF → structured text, metadata, references
Object storage	MinIO	Generated images, cached PDFs, book assets
Observability	Bull Board, Grafana + Loki, webhook alerts	Metrics, logs, quality dashboards

Table 1 — Core components.

4.1 Why a durable job queue (BullMQ on Redis)

A FACTOR run is a long, failure-prone chain of external calls (retrieval, several LLM steps, image generation, publishing) that can take minutes and hit API timeouts, rate limits, and transient errors. Running such a pipeline as in-process threads or a naive cron script would lose all work on a crash and offer no backpressure. FACTOR therefore models every step as a persisted job on **BullMQ**, backed by **Redis**:

- **Durability & retries.** Jobs and their state live in Redis, not in a worker's memory. A crashed or restarted worker resumes where it left off; failed steps retry with exponential backoff instead of failing the whole piece.
- **Backpressure & rate limiting.** LLM and embedding endpoints have rate and cost ceilings. BullMQ's global concurrency and rate limiters throttle work across all workers, preventing quota exhaustion and runaway spend.
- **Scheduling.** Repeatable (cron-like) jobs drive daily topic selection per workspace, decoupled from any single always-on process; delayed jobs handle scheduled publishing.
- **Horizontal scale & fairness.** The pipeline is I/O-bound (mostly waiting on models); stateless workers pull from shared queues, so throughput scales by adding workers. Per-workspace queues and priorities stop one backlog from starving others.
- **Idempotency & observability.** Job keys prevent double-processing; the job lifecycle feeds the immutable run_events audit trail (tokens, cost, latency per step) and a live Bull Board dashboard.
- **Reuses existing infrastructure.** The target deployment already runs Redis for its CMS, so orchestration adds no new datastore — only queues on the operator's own Redis.

5. Knowledge Layer

5.1 Topic Workspaces

A **Topic Workspace** is the unit of multi-tenancy. Each workspace defines: a topic backlog (imported or AI-expanded, user-curated), one or more source connectors, an editorial profile (audience, tone, languages, content-type mix), credibility policy (minimum source tier per content type), and a publishing target (CMS database, REST/GraphQL API, webhook, or static export).

5.2 Source connectors

Connector	Protocol	Typical sources
Drive repository	Google Drive API / WebDAV / S3	Curated PDFs, books, internal documents
Journal harvester	OAI-PMH, Crossref, RSS/Atom	Open-access scientific journals (e.g., OJS instances)
Official web sources	Sitemap crawl + change detection	Government portals, standards bodies, official docs
Book repository	EPUB/PDF ingestion from MinIO	Licensed reference books, textbooks, manuals
Manual upload	Admin UI	One-off authoritative documents

Table 2 — Source connectors per workspace.

All connectors feed one ingestion pipeline: parse (GROBID for scholarly PDFs) → semantic chunking (500–800 tokens, 100 overlap) → embedding → pgvector, with mandatory metadata: title, authors, year, DOI/URL, source type, and a 1–5 credibility score (peer-reviewed guideline = 5, vendor blog = 2). A relevance classifier (small LLM) filters harvested items before they enter the corpus. Topics whose corpus support falls below a workspace-defined threshold are never scheduled — the first anti-hallucination gate fires before any text is generated.

6. Agent Orchestration

Each content piece is one pipeline run through an explicit state machine:

```
QUEUED → PLANNING → RESEARCHING → OUTLINING → DRAFTING
→ FACT_CHECKING → BIAS_REVIEW → REVISING (loop ≤ 3)
→ TRANSLATING → TRANSLATION_QA → META_SEO → IMAGE_GEN
→ [HUMAN_REVIEW] → PUBLISHING → PUBLISHED   (else: REJECTED / NEEDS_ATTENTION)
```

Agent	Model tier	Responsibility
Planner	Small/medium	Selects daily topics per workspace: priority, category rotation, seasonality, corpus support, dedup vs. published content
Researcher	Retrieval + rerank	Hybrid search (pgvector + full-text) over workspace corpus; produces a citable research pack (top ~20–30 chunks)
Outliner	Medium	Section structure with key claims mapped to source chunk IDs; genre template (article/review/tutorial/opinion)
Writer	Medium	Drafts in primary language; every factual claim carries an inline citation marker [[chunk:id]]; genre-specific rules
Fact-checker	Largest available	Independent context; verdict per claim: supported / partial / unsupported / contradicted. Hard gate
Bias & safety reviewer	Medium	Commercial and demographic bias, overclaiming, risk-benefit balance, regulatory compliance per domain profile
Translator + QA	Medium	Transcreation into target languages after verification; cross-lingual QA ensures zero claim drift

Agent	Model tier	Responsibility
Meta & SEO	Small	Titles, descriptions, slugs, tags, JSON-LD schema, reading time, Open Graph — per language
Image agent	Image model	Brand-consistent illustration prompt + generation; alt text per language; no photorealistic people in sensitive domains
Publisher	Deterministic	Transactional injection into the target CMS/database: content, translations, citations, images, meta; scheduled release

Table 3 — Agent roles. Revision loop is capped at 3 iterations; overflow escalates to a human queue.

6.1 Content types and grounding rules

Type	Subjectivity	Grounding rule
Article (informational/scientific)	None	Every claim cited; minimum average credibility per workspace policy
Tutorial / how-to	Low	Steps validated against official documentation versions; version pinning recorded
Review	Moderate	Factual attributes (specs, prices, study results) cited; judgments must follow from cited facts
Opinion piece	High	Labeled as opinion; premises cited; counter-arguments required by the bias reviewer

Table 4 — Genre-specific grounding policy.

7. Anti-Hallucination Gate System

#	Gate	Mechanism	On failure
1	Source sufficiency	≥ N relevant chunks, minimum credibility average	Run aborted; topic flagged
2	Grounded writing	Writer constrained to research pack; mandatory citation markers	By construction
3	Independent fact-check	Separate model/context compares claims to original chunks	Revision (max 3)
4	Bias & ethics review	Commercial/demographic bias, overclaiming, domain regulations	Revision
5	Cross-lingual QA	Claim-level diff between source and translated versions	Re-translate
6	Schema validation	Structured (JSON) outputs validated at every step; unique slugs	Step retry
7	Human review	Per-workspace approval queue; autonomy earned by pass-rate	Editor fixes/rejects
8	Post-publish audit	Weekly sampling + public inaccuracy-report channel	Unpublish + prompt tuning

Table 5 — Eight-layer gate system. Every step is logged to an immutable audit trail (run_events).

Additional controls: low sampling temperature (0.2–0.4) for writing and verification; prompt caching of system prompts and research packs; per-run cost budgets with automatic alerts; idempotency keys on every pipeline step.

8. Data Model (summary)

Table	Purpose
workspaces	Topic workspace: domain, languages, editorial & credibility policy, publishing target

Table	Purpose
topics	Per-workspace backlog: slug, titles, category, priority, corpus support count, last published
sources / source_chunks	Corpus documents and embedded chunks (pgvector HNSW + tsvector full-text)
journal_endpoints / connectors	Harvest endpoints and connector configs per workspace
pipeline_runs / run_events	State machine instance per piece; immutable audit trail with tokens, cost, latency
articles / article_translations	Published output: rich text (portable JSON/HTML), per-locale metadata
article_citations	Claim → source chunk mapping with verification verdicts
article_images	Generated assets, prompts, models, alt text per locale

Table 6 — Core relations. Full DDL is maintained in the repository.

9. Deployment Profiles & Hardware Requirements

9.1 Hybrid profile (external LLM APIs)

Orchestration, data, parsing, and storage are self-hosted; LLM, embedding, and image inference use external APIs. Lowest hardware cost, highest verification quality (frontier fact-checking models).

Service	vCPU	RAM	Notes
Orchestrator API + Bull Board	0.5	512 MB	
Workers ×2	1.0	1 GB	I/O-bound
PostgreSQL + pgvector	2.0	4 GB	HNSW index resident in RAM
Redis	0.25	512 MB	
MinIO	0.25	512 MB	
GROBID (optional, on-demand)	2.0	4 GB	Run during ingest windows only
PaaS + OS overhead	1.0	2 GB	
Recommended VPS	6 vCPU	16 GB	100 GB SSD

Table 7 — Hybrid profile sizing. Minimal viable: 4 vCPU / 8 GB / 60 GB SSD without GROBID.

9.2 Fully self-hosted profile on AMD Instinct (sovereign inference)

All inference runs locally on AMD GPUs via the ROCm stack. The reference deployment runs **Google Gemma 3** (served by **vLLM on ROCm**) for the Writer and Fact-checker and **Google EmbeddingGemma** for retrieval — so generation and semantic search both run on the Gemma model family — with SDXL image generation on the same device. The target is a single **AMD Instinct MI300X** (CDNA 3, gfx942) with **192 GB of HBM3**: its exceptionally large memory lets one GPU host models up to a 70B-class instruction model at full precision — no quantization, no multi-GPU sharding — while co-hosting embeddings and image generation. Because open models verify facts less reliably than frontier models, gates 3 and 7 are tightened (dual-pass fact-checking; mandatory human review for high-stakes workspaces).

This sovereign-inference profile is not limited to data-center hardware. For the AMD Developer Hackathon it runs **live on a single AMD Radeon PRO W7900** (RDNA 3, gfx1100, 48 GB) using **llama.cpp (ROCm/HIP)** to serve **Gemma 3** (up to the 27B instruction model), with **bge-m3** embeddings and **SDXL-Turbo** image generation co-hosted on the same card — demonstrating that the profile scales down to a workstation-class AMD GPU, and up to a 192 GB Instinct MI300X for larger models, with no change to the application (an OpenAI-compatible endpoint in both cases).

Service	Requirement	Notes
vLLM (70B, bf16)	1× AMD Instinct MI300X (192 GB HBM3), ROCm 6/7	Fits a 70B model unquantized on a single GPU; OpenAI-compatible API
Embeddings (bge-m3)	Co-hosted on the same MI300X (vLLM /v1/embeddings)	No separate embedding server required
ComfyUI (SDXL/Flux)	Co-hosted on the MI300X via ROCm	Batch image generation off-peak
Host (AMD Developer Cloud, typical)	MI300X · ~20 vCPU · 235 GB RAM · ~700 GB NVMe	Single-node deployment; no sharding complexity

Table 8 — Fully self-hosted sizing on AMD Instinct MI300X. A single MI300X replaces the multi-GPU rigs that comparable 70B deployments require on 24–48 GB cards.

9.3 Indicative operating cost

Profile	Infra / month	Inference / month (5 pieces/day, bilingual)	Total
Hybrid	US$ 40–80 (VPS)	US$ 130–160 (hosted LLM API)	US$ 170–240
Self-hosted on AMD MI300X	AMD Instinct GPU (owned or cloud-rented)	US$ 0 marginal (no per-token cost)	GPU cost only

Table 9 — Indicative costs; verify current pricing before budgeting. On AMD, inference cost is fixed (GPU) rather than per-token, so unit economics improve with volume.

10. Security, Compliance & Transparency

- **Data sovereignty.** Corpus, drafts, and audit trails never leave the deployment in self-hosted mode; in hybrid mode only research-pack excerpts are transmitted to inference APIs.
- **Licensing.** Book and journal ingestion respects source licenses; the corpus registry records license terms per source, and the publisher can enforce quote-length limits.
- **AI disclosure.** Published pieces carry configurable AI-assisted attribution and a reviewed-by label, aligned with emerging disclosure norms and platform policies.
- **Auditability.** Every claim in every published piece is reconstructible: claim → chunk → source document → verdict, with model, tokens, and timestamps in run_events.
- **Access control.** Workspace-scoped RBAC; publishing credentials stored per workspace; secrets in the PaaS vault.

11. Current Implementation Status

This section states plainly what exists today versus what the architecture above specifies for production, so the reader can separate demonstrated capability from design intent.

11.1 What is proven

- **End-to-end pipeline (interactive demo).** A working application drives the full state machine — Planner through Publisher, all eight gates, the revision loop, bilingual output, and click-through citation tracing — with three canonical scenarios (grounded happy path, a fact-check failure that triggers revision, and a weak-corpus topic rejected at Gate 1).
- **Real inference on AMD GPUs, powered by Google Gemma.** The Writer, Fact-checker and Translator run live with **Gemma 3** behind an OpenAI-compatible endpoint; the Researcher performs real semantic retrieval; and the Image agent generates featured images with SDXL — all on AMD hardware, producing grounded, correctly-cited drafts and per-claim verdicts end-to-end. This is **live now on an AMD Radeon**

PRO W7900 (RDNA 3, gfx1100, ROCm 7.2) serving **Gemma-3-27B** via llama.cpp with **bge-m3** retrieval and SDXL-Turbo images, and it was **previously proven on an AMD Instinct MI300X** (gfx942, ROCm 7.2.4) serving Gemma 3 via vLLM with **EmbeddingGemma** — both validating the fully self-hosted, sovereign-inference profile on AMD using the Gemma family for generation and retrieval.

• **Alternative hosted engine.** The same pipeline also runs against a **Fireworks AI** backend (**gpt-oss-120b** for writing and fact-checking, **qwen3-embedding-8b** for retrieval), selectable per run — demonstrating the application is model- and provider-agnostic over any OpenAI-compatible endpoint.

11.2 What is still simulated

The present demo uses a small, curated **seed corpus** and canonical outputs for the non-inference steps; it is not yet wired to the operator's live data. Concretely, the following production integrations remain to be built:

Capability	Status	Effort to production
Inference on AMD GPU: Writer, Fact-checker, Translator, Researcher, Image	Working (live)	Done — W7900 live; MI300X proven
Full pipeline flow, gates, bilingual, citations (demo)	Working (simulated data)	Done
Topic backlog (hundreds–thousands of real topics)	Not connected	Low — import to DB (2–3 days)
Corpus: Google Drive + journals + books → pgvector	Not connected	High — connectors, parsing, embedding (~3 weeks)
Publish into the content/article database (CMS)	Not connected	Medium — transactional injector (3–5 days)

Table 10 — Honest capability matrix. Inference on AMD is real; corpus and publishing integrations are the remaining build.

A pragmatic first milestone is a **vertical slice** (about one to two weeks): ingest a real subset of the operator's corpus, embed it on the same MI300X, point retrieval at the resulting pgvector store, load the real topic backlog, and produce a handful of genuinely grounded articles — optionally written to a single database table as proof of publishing. This converts the demonstration from a simulation into a working system on real data, on AMD, without first building the entire production service.

12. Implementation Roadmap

Phase	Weeks	Deliverables
1 — Corpus foundation	1–2	pgvector schema, connectors (Drive + OAI-PMH), chunking & embedding, topic import, support scoring
2 — Core pipeline	3–4	State machine on BullMQ; Planner → Writer → Fact-checker; first end-to-end cited article
3 — Quality & multilingual	5–6	Bias reviewer, revision loop, translation + QA, meta/SEO, image generation, transactional publishing, review UI
4 — Multi-workspace & autonomy	7–9	Workspace model & RBAC, daily scheduling, dashboards, post-publish audit, per-workspace autonomy graduation
5 — Self-hosted inference (optional)	10–12	vLLM / llama.cpp / ComfyUI deployment profile, dual-pass verification, benchmark vs. hybrid quality
6 — Domain model fine-tune (optional)	12+	LoRA SFT of a compact writer model on curated grounded articles; DPO for citation completeness. Off-the-shelf instruction models are used until then

Table 11 — Roadmap to production. Phase 5 (self-hosted AMD inference) is already validated on AMD (Radeon PRO W7900 live; Instinct MI300X proven); the near-term critical path is Phase 1 (corpus foundation).

12.1 Optional: domain model fine-tuning (future work)

FACTOR steers models through prompting, retrieval, and gates — it performs **no model training** today and uses off-the-shelf instruction-tuned models (for example gemma-3-27b-*it*, already SFT- and preference-aligned by its makers). Training is therefore *not* on the critical path. As an optional future enhancement, a small domain model could be adapted on the same AMD GPU that already serves inference:

- **SFT (LoRA / QLoRA).** Supervised fine-tuning of a compact model (e.g., Gemma-3-4B) on a curated set of grounded, correctly-cited articles, to internalize house style and citation discipline. Feasible on a single AMD Radeon / Instinct GPU; the dominant cost is dataset curation, not compute.
- **DPO (Direct Preference Optimization).** Preference tuning on pairs of grounded-and-cited versus ungrounded drafts, biasing the Writer toward citation-complete output and reducing revision-loop iterations.
- Online reinforcement methods (PPO, GRPO, RLOO) are deliberately **out of scope**: they add substantial complexity, memory, and training instability for marginal benefit over FACTOR's retrieval-and-gate approach.

Any fine-tuning would **complement, never replace**, the retrieval-first architecture: grounding and independent fact-checking remain the primary guarantees against hallucination. A tuned model that still cannot cite a claim would be rejected by the same gates as an untuned one.

13. Conclusion

FACTOR treats factual integrity as an architectural property, not an aspiration. By binding generation to a curated, user-owned corpus, separating writing from verification, and gating every piece through eight auditable checks, it makes autonomous content production safe enough for the domains where accuracy matters most - while remaining fully self-hostable, topic-agnostic, and economically viable at a cadence of dozens of verified, multilingual pieces per week.